

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2

По дисциплине: «Модели решения задач в ИС»

Тема: «Многослойный персептрон для задачи «исключающее ИЛИ» с числовыми классами: сравнение функций потерь MSE и Binary Cross-Entropy»

Выполнил:

Студент 4 курса

Группы ИИ-26

Шоева Е.Т.

Проверил:

Никитин С.К.

Цель работы: На основе многослойного персептрона архитектуры 2-2-1, реализованного в Лабораторной работе №1 для задачи XOR с числовыми классами c_0 и c_1 из диапазона $[-10; 10]$, реализовать альтернативную функцию потерь – бинарную кросс-энтропию (Binary Cross-Entropy, BCE) – и провести сравнительный анализ обучения сети с MSE (Лабораторная работа №1) и с BCE по скорости сходимости, качеству обучения и устойчивости к выбору начальных весов.

Ход работы

Задачи лабораторной работы

1. Взять за основу код многослойного персептрона 2-2-1 из Лабораторной работы №1.
 2. Поскольку BCE определена для выходов в диапазоне $(0; 1)$, а целевые значения в вашем варианте – произвольные числа c_0 и c_1 из диапазона $[-10; 10]$, реализовать нормализацию (масштабирование) целевых значений перед обучением и обратное преобразование выхода сети после обучения:
 - привести c_0 и c_1 к бинарным меткам 0 и 1;
 - обучать сеть на нормализованных целевых значениях с функцией потерь BCE;
 - при выводе результата преобразовывать выход сети $\hat{y} \in (0; 1)$ обратно в шкалу $[c_0; c_1]$.
 3. Реализовать корректный градиент для связки BCE + сигмоида на выходном слое; для скрытого слоя градиент передаётся по стандартному правилу цепочки.
 4. Обучить сеть на полной таблице истинности своего варианта (4 примера) в двух конфигурациях:
 - **Конфигурация А (базовая, из ЛР №1):** MLP 2-2-1, функция потерь MSE, целевые значения – исходные c_0/c_1 , выходной слой – сигмоида. Результаты можно взять из отчёта по ЛР №1 или, при необходимости, повторить обучение с фиксацией всех метрик.
 - **Конфигурация Б (новая):** MLP 2-2-1, функция потерь BCE, целевые значения – нормализованные $(0/1)$, с последующим пересчётом выхода в шкалу $[c_0; c_1]$.
- Критерий остановки для обеих конфигураций одинаков – суммарная ошибка $E_s \leq E_e$ (используйте то же значение E_e , что и в ЛР №1, либо согласуйте новое значение отдельно для MSE и для BCE, поскольку это разные по своей природе величины – обоснуйте выбор в отчёте).
5. Для каждой конфигурации зафиксировать:
 - количество эпох до достижения критерия остановки (либо факт недостижения за максимально допустимое число эпох);
 - итоговую суммарную ошибку;
 - ассурасу на всех 4 примерах таблицы истинности;
 - среднюю абсолютную ошибку в исходной шкале $[c_0; c_1]$.
 6. Провести не менее 5 запусков обучения для каждой конфигурации с разной случайной инициализацией весов (менять seed) и сравнить устойчивость сходимости: в скольких из 5 запусков сеть достигла критерия остановки, и какой разброс по числу эпох наблюдается.
 7. Построить общий график зависимости суммарной ошибки от номера эпохи для конфигурации А (MSE) и конфигурации Б (BCE) на одних осях (для представительного запуска каждой конфигурации).
 8. Реализовать блок визуализации результатов, включающий:
 - **График сходимости** – суммарная ошибка E_s от номера эпохи, конфигурации А и Б на одних осях, разными цветами/стилями линий, с обозначением порога E_e .
 - **Диаграмму разброса числа эпох по 5 запускам** (bar chart или scatter) для конфигурации А и конфигурации Б – наглядно показывающую устойчивость сходимости из п. 6 (например, столбики числа эпох для каждого из 5 seed, сгруппированные по конфигурации; запуски, не достигшие критерия остановки, отметить отдельно, например штриховкой или отдельным цветом).
 - **Визуализацию разделяющей поверхности скрытого слоя** – для представительного обученного экземпляра каждой конфигурации построить на плоскости (А, В) карту выходного значения сети (heatmap или contour plot) с наложенными точками обучающей выборки (4 примера таблицы истинности), закрашенными по истинному

классу. Диапазон осей А и В – $[-10; 10]$ (весь допустимый диапазон входных значений). Для конфигурации Б цветовую шкалу можно строить как по нормализованному $\hat{y} \in (0; 1)$, так и по пересчитанному значению в шкале $[c0; c1]$ – выбрать более наглядный вариант и обосновать выбор.

- **Диаграмму сравнения точности восстановления шкалы** – столбчатую диаграмму средней абсолютной ошибки для конфигурации А и конфигурации Б, чтобы наглядно сопоставить точность регрессии к исходной шкале $[c0; c1]$.

Каждый график должен иметь подписанные оси, заголовок и легенду (где применимо). Разделяющую поверхность и графики точности рекомендуется разместить рядом (например, subplot 2×2 или отдельными рисунками) для удобства сравнения в отчёте.

9. Реализовать режим функционирования сети: пользователь вводит пару чисел (А, В) из диапазона $[-10; 10]$ (не обязательно совпадающих с $c0$ и $c1$). Сеть должна выводить:

- выход в нормализованной шкале $\hat{y} \in (0; 1)$ (с точностью до 4 знаков);
- выход, пересчитанный в исходную шкалу $[c0; c1]$ по формуле из п. 2;
- к какому классу ($c0$ или $c1$) ближе полученное значение.

Продемонстрировать работу режима функционирования на всех 4 примерах исходной таблицы истинности и не менее чем на 2–3 дополнительных парах (А, В), не входящих в обучающую выборку.

Вариант:

Вариант 11 ($c0 = 5, c1 = -2$)

А	В	$A \oplus B$
5	5	5
5	-2	-2
-2	5	-2
-2	-2	5

Архитектура

MLP 2-2-1: 2 входных нейрона (А, В) → 2 нейрона скрытого слоя → 1 выходной нейрон (С).

Сигмоида на всех слоях. Веса — малые случайные значения. Обучение — backprop, онлайн-режим (обновление после каждого примера).

SLP (однослойный): 2 входа напрямую соединены с одним выходным сигмоидным нейроном, без скрытого слоя.

Нормализация (общая для обеих конфигураций ЛР №2): $c0 \rightarrow 1, c1 \rightarrow 0$ перед обучением; выход $\hat{y} \in (0,1)$ пересчитывается обратно по формуле $y_{\text{real}} = c1 + \hat{y} \cdot (c0 - c1)$.

Отличие конфигураций ЛР №2:

- **MSE:** $\text{delta_out} = (\text{target} - \hat{y}) \cdot \hat{y} \cdot (1 - \hat{y})$
- **BCE:** $\text{delta_out} = \text{target} - \hat{y}$ (без домножения на производную сигмоиды — градиент не насыщается)

`import numpy as np`

`import matplotlib.pyplot as plt`

$C0 = 5$

$C1 = -2$

`RAW_DATA = [`

`(C0, C0, C0),`

`(C0, C1, C1),`

`(C1, C0, C1),`

`(C1, C1, C0),`

`]`

```

LEARNING_RATE = 0.5
EE_MSE = 0.01
EE_BCE = 0.05
MAX_EPOCHS = 10000
SEEDS = [1, 2, 3, 4, 5]
REPRESENTATIVE_SEED = 1
EPS = 1e-9
def encode_value(x):
    return 1.0 if x == C0 else 0.0
def decode_value_continuous(y):
    return C1 + y * (C0 - C1)

def decode_class(y):
    return C0 if y >= 0.5 else C1
def encode_arbitrary(x):
    return 1.0 if abs(x - C0) < abs(x - C1) else 0.0
def encode_continuous(x):
    return (x - C1) / (C0 - C1)
def build_training_set():
    X, Y = [], []
    for a, b, target in RAW_DATA:
        X.append([encode_value(a), encode_value(b)])
        Y.append([encode_value(target)])
    return np.array(X), np.array(Y)
def sigmoid(x):
    return 1.0 / (1.0 + np.exp(-x))
def sigmoid_derivative(y):
    return y * (1.0 - y)
class MLP221:
    def __init__(self, loss_type="mse", lr=LEARNING_RATE, seed=0):
        assert loss_type in ("mse", "bce")
        self.loss_type = loss_type
        self.lr = lr
        rng = np.random.default_rng(seed)
        self.W1 = rng.uniform(-0.5, 0.5, size=(2, 2))
        self.b1 = rng.uniform(-0.5, 0.5, size=(2,))
        self.W2 = rng.uniform(-0.5, 0.5, size=(2, 1))
        self.b2 = rng.uniform(-0.5, 0.5, size=(1,))

    def forward(self, x):
        hidden_in = x @ self.W1 + self.b1
        hidden_out = sigmoid(hidden_in)
        out_in = hidden_out @ self.W2 + self.b2
        out = sigmoid(out_in)
        return hidden_out, out

    def loss_and_delta_out(self, target, out):

        if self.loss_type == "mse":
            error = target - out
            loss = float(np.sum(error ** 2))

```

```

        delta_out = error * sigmoid_derivative(out)
    else:
        y = np.clip(out, EPS, 1 - EPS)
        loss = float(-np.sum(target * np.log(y) + (1 - target) * np.log(1 - y)))

        delta_out = target - out
    return loss, delta_out

def train_step(self, x, target):
    hidden_out, out = self.forward(x)
    loss, delta_out = self.loss_and_delta_out(target, out)

    error_hidden = delta_out @ self.W2.T
    delta_hidden = error_hidden * sigmoid_derivative(hidden_out)

    self.W2 += self.lr * np.outer(hidden_out, delta_out)
    self.b2 += self.lr * delta_out
    self.W1 += self.lr * np.outer(x, delta_hidden)
    self.b1 += self.lr * delta_hidden

    return loss

def predict(self, x):
    _, out = self.forward(np.array(x, dtype=float))
    return float(out[0])

def train(self, X, Y, ee, max_epochs=MAX_EPOCHS):
    history = []
    Es = float("inf")
    for epoch in range(1, max_epochs + 1):
        Es = 0.0
        for i in range(len(X)):
            Es += self.train_step(X[i], Y[i])
        history.append(Es)
        if Es <= ee:
            return epoch, Es, history, True
    return max_epochs, Es, history, False

def accuracy(model, X, Y):
    correct = 0
    for i in range(len(X)):
        pred_class = 1.0 if model.predict(X[i]) >= 0.5 else 0.0
        if pred_class == Y[i][0]:
            correct += 1
    return correct / len(X)

def mae_real_scale(model, X):
    errors = []
    for x, (a, b, target) in zip(X, RAW_DATA):

```

```

    y_hat = model.predict(x)
    pred_real = decode_value_continuous(y_hat)
    errors.append(abs(pred_real - target))
return float(np.mean(errors))

```

```

def run_series(loss_type, ee, X, Y, seeds=SEEDS):
    results = []
    for seed in seeds:
        model = MLP221(loss_type=loss_type, seed=seed)
        epochs, Es, history, converged = model.train(X, Y, ee=ee)
        results.append({
            "seed": seed,
            "epochs": epochs,
            "Es": Es,
            "converged": converged,
            "accuracy": accuracy(model, X, Y),
            "mae": mae_real_scale(model, X),
            "model": model,
            "history": history,
        })
    return results

```

```

def print_series_table(title, results):
    print(f"\n{title}")
    print(f"{'seed':>6}{'epochs':>10}{'converged':>12}{'Es':>14}{'accuracy':>10}{'MAE':>10}")
    for r in results:
        print(f"{r['seed']:>6}{r['epochs']:>10}{str(r['converged']):>12}"
              f"{r['Es']:>14.6f}{r['accuracy']:>10.2f}{r['mae']:>10.4f}")

```

```

def plot_convergence(hist_a, hist_b, filename="conv_plot.png"):
    plt.figure(figsize=(7, 5))
    plt.plot(range(1, len(hist_a) + 1), hist_a, label="Конфигурация А (MSE)", color="tab:blue")
    plt.plot(range(1, len(hist_b) + 1), hist_b, label="Конфигурация Б (BCE)", color="tab:orange")
    plt.axhline(EE_MSE, color="tab:blue", linestyle="--", linewidth=0.8, label=f"Ee (MSE) = {EE_MSE}")
    plt.axhline(EE_BCE, color="tab:orange", linestyle="--", linewidth=0.8, label=f"Ee (BCE) = {EE_BCE}")
    plt.title("Сходимость: суммарная ошибка Es по эпохам (MSE vs BCE)")
    plt.xlabel("Эпоха")
    plt.ylabel("Суммарная ошибка Es")
    plt.legend()
    plt.tight_layout()
    plt.savefig(filename, dpi=150)
    plt.close()
    print(f"Сохранён график сходимости: {filename}")

```

```

def plot_epochs_bar(results_a, results_b, filename="epochs_bar.png"):

```

```

seeds = [r["seed"] for r in results_a]
n = len(seeds)
x = np.arange(n)
width = 0.35

```

```

fig, ax = plt.subplots(figsize=(7, 5))
for i, r in enumerate(results_a):
    color = "tab:blue" if r["converged"] else "none"
    edge = "tab:blue"
    hatch = None if r["converged"] else "/"
    ax.bar(x[i] - width / 2, r["epochs"], width, color=color, edgecolor=edge, hatch=hatch)
for i, r in enumerate(results_b):
    color = "tab:orange" if r["converged"] else "none"
    edge = "tab:orange"
    hatch = None if r["converged"] else "/"
    ax.bar(x[i] + width / 2, r["epochs"], width, color=color, edgecolor=edge, hatch=hatch)

```

```

from matplotlib.patches import Patch
legend_elems = [
    Patch(facecolor="tab:blue", label="MSE, сошлось"),
    Patch(facecolor="none", edgecolor="tab:blue", hatch="/", label="MSE, не сошлось"),
    Patch(facecolor="tab:orange", label="BCE, сошлось"),
    Patch(facecolor="none", edgecolor="tab:orange", hatch="/", label="BCE, не сошлось"),
]
ax.legend(handles=legend_elems)
ax.set_xticks(x)
ax.set_xticklabels([f"seed={s}" for s in seeds])
ax.set_xlabel("Запуск (seed)")
ax.set_ylabel("Число эпох до сходимости")
ax.set_title("Устойчивость сходимости: 5 запусков, MSE vs BCE")
plt.tight_layout()
plt.savefig(filename, dpi=150)
plt.close()
print(f"Сохранена диаграмма разброса эпох: {filename}")

```

```

def plot_decision_surface(model_a, model_b, filename="surface_plot.png"):
    grid = np.linspace(-10, 10, 200)
    AA, BB = np.meshgrid(grid, grid)

```

```

def surface(model):
    Z = np.zeros_like(AA)
    for i in range(AA.shape[0]):
        for j in range(AA.shape[1]):
            x = [encode_continuous(AA[i, j]), encode_continuous(BB[i, j])]
            y_hat = model.predict(x)
            Z[i, j] = decode_value_continuous(y_hat)
    return Z

```

```

Za = surface(model_a)

```

```

Zb = surface(model_b)

fig, axes = plt.subplots(1, 2, figsize=(12, 5.2))
for ax, Z, title in zip(axes, [Za, Zb], ["Разделяющая поверхность (MSE)", "Разделяющая
поверхность (BCE)"]):
    im = ax.contourf(AA, BB, Z, levels=25, cmap="coolwarm_r")
    ax.contour(AA, BB, Z, levels=25, colors="k", linewidths=0.3, alpha=0.5)
    fig.colorbar(im, ax=ax, label="Выход сети (шкала [c1; c0])")
    pts_a = [p[0] for p in RAW_DATA]
    pts_b = [p[1] for p in RAW_DATA]
    ax.scatter(pts_a, pts_b, s=110, c="crimson", edgecolors="k", zorder=5, label="Точки XOR")
    ax.set_xlabel("Вход А")
    ax.set_ylabel("Вход В")
    ax.set_title(title)
    ax.legend(loc="upper right")
plt.suptitle("Визуализация разделяющей поверхности для обеих конфигураций")
plt.tight_layout()
plt.savefig(filename, dpi=150)
plt.close()
print(f"Сохранена визуализация разделяющей поверхности: {filename}")

def plot_mae_bar(mae_a, mae_b, filename="mae_bar.png"):
    plt.figure(figsize=(5, 4.5))
    bars = plt.bar(["MSE (конфиг. А)", "BCE (конфиг. Б)"], [mae_a, mae_b],
                    color=["tab:blue", "tab:orange"])
    for b in bars:
        h = b.get_height()
        plt.text(b.get_x() + b.get_width() / 2, h, f"{h:.4f}", ha="center", va="bottom")
    plt.ylabel("Средняя абсолютная ошибка (шкала [c1; c0])")
    plt.title("Точность восстановления исходной шкалы")
    plt.tight_layout()
    plt.savefig(filename, dpi=150)
    plt.close()
    print(f"Сохранена диаграмма MAE: {filename}")

def run_and_print(model, a, b, label):
    x = [encode_arbitrary(a), encode_arbitrary(b)]
    y_hat = model.predict(x)
    y_real = decode_value_continuous(y_hat)
    cls = decode_class(y_hat)
    print(f" {label:<28} A={a:>5} B={b:>5} | вход={x} | "
          f"y_hat={y_hat:.4f} | y_real={y_real:>7.3f} | ближе к классу {'c0' if cls == C0 else 'c1'}
          ({cls})")

def demo_examples(model, model_name):
    print(f"\n--- Демонстрация работы сети «{model_name}» ---")
    print("На 4 примерах обучающей выборки:")
    for a, b, target in RAW_DATA:
        run_and_print(model, a, b, f"target={target}")
    print("На дополнительных парах (вне обучающей выборки):")

```

```

for a, b in [(3, 7), (-5, -8), (0, 0)]:
    run_and_print(model, a, b, "вне выборки")

def interactive_mode(model, model_name):
    print(f"\n--- Режим работы обученной сети ({model_name}) ---")
    print("Введите пару чисел A, B из диапазона [-10, 10] (или 'q' для выхода).")
    while True:
        raw = input("Введите A B через пробел: ").strip()
        if raw.lower() in ("q", "quit", "exit", ""):
            print("Завершение режима ввода.")
            break
        try:
            parts = raw.replace(",", " ").split()
            a, b = float(parts[0]), float(parts[1])
        except (ValueError, IndexError):
            print("Некорректный ввод. Пример: 3.5 -7")
            continue
        if not (-10 <= a <= 10 and -10 <= b <= 10):
            print("Значения должны быть в диапазоне [-10, 10].")
            continue
        run_and_print(model, a, b, "ввод пользователя")

def main():
    X, Y = build_training_set()
    print("Обучающая выборка (закодированная):")
    for x, y, (a, b, t) in zip(X, Y, RAW_DATA):
        print(f" A={a:>3} B={b:>3} -> вход={x} target={y} (реальный={t})")

    print("\n" + "=" * 70)
    print("Серия из 5 запусков — Конфигурация А (MSE)")
    print("=" * 70)
    results_a = run_series("mse", EE_MSE, X, Y)
    print_series_table("Результаты (MSE):", results_a)

    print("\n" + "=" * 70)
    print("Серия из 5 запусков — Конфигурация Б (BCE)")
    print("=" * 70)
    results_b = run_series("bce", EE_BCE, X, Y)
    print_series_table("Результаты (BCE):", results_b)

    conv_a = sum(r["converged"] for r in results_a)
    conv_b = sum(r["converged"] for r in results_b)
    epochs_a_conv = [r["epochs"] for r in results_a if r["converged"]]
    epochs_b_conv = [r["epochs"] for r in results_b if r["converged"]]

    print("\n" + "=" * 70)
    print("Сводка по устойчивости сходимости")
    print("=" * 70)

```

```

print(f"MSE: сошлось {conv_a}/5 запусков; "
      f"эпох (сошедшиеся): {epochs_a_conv} "
      f"(среднее={np.mean(epochs_a_conv) if epochs_a_conv else float('nan'):.1f})")
print(f"BCE: сошлось {conv_b}/5 запусков; "
      f"эпох (сошедшиеся): {epochs_b_conv} "
      f"(среднее={np.mean(epochs_b_conv) if epochs_b_conv else float('nan'):.1f})")

def pick_representative(results):
    converged = [r for r in results if r["converged"]]
    if converged:
        return min(converged, key=lambda r: r["epochs"])
    return min(results, key=lambda r: r["Es"])

res_a_rep = pick_representative(results_a)
res_b_rep = pick_representative(results_b)
model_a, hist_a = res_a_rep["model"], res_a_rep["history"]
model_b, hist_b = res_b_rep["model"], res_b_rep["history"]

print("\n" + "=" * 70)
print(f"Итоговая таблица (представительные запуски: "
      f"MSE seed={res_a_rep['seed']}, BCE seed={res_b_rep['seed']}")
print("=" * 70)
print(f"{'Критерий':<35}{'Конфиг. А (MSE)':>18}{'Конфиг. Б (BCE)':>18}")
print(f"{'Эпох до сходимости':<35}{res_a_rep['epochs']:>18}{res_b_rep['epochs']:>18}")
print(f"{'Критерий Es<=Ee достигнут':<35}{str(res_a_rep['converged']):>18}{str(res_b_rep['converged']):>18}")
print(f"{'Итоговая Es':<35}{res_a_rep['Es']:>18.6f}{res_b_rep['Es']:>18.6f}")
print(f"{'Accuracy':<35}{res_a_rep['accuracy']:>18.2f}{res_b_rep['accuracy']:>18.2f}")
print(f"{'MAE (шкала c0/c1)':<35}{res_a_rep['mae']:>18.4f}{res_b_rep['mae']:>18.4f}")

plot_convergence(hist_a, hist_b)
plot_epochs_bar(results_a, results_b)
plot_decision_surface(model_a, model_b)
plot_mae_bar(res_a_rep["mae"], res_b_rep["mae"])

demo_examples(model_a, "Конфигурация А (MSE)")
demo_examples(model_b, "Конфигурация Б (BCE)")

print("\nВыберите сеть для интерактивного режима:")
print(" 1 - Конфигурация А (MSE)")
print(" 2 - Конфигурация Б (BCE)")
choice = input("Ваш выбор (1/2, Enter для пропуска): ").strip()
if choice == "1":
    interactive_mode(model_a, "Конфигурация А (MSE)")
elif choice == "2":
    interactive_mode(model_b, "Конфигурация Б (BCE)")
else:
    print("Интерактивный режим пропущен.")

```

```
if __name__ == "__main__":
    main()
```

```
Обучающая выборка (закодированная):
A= 5 B= 5 -> вход=[1. 1.] target=[1.] (реальный=5)
A= 5 B= -2 -> вход=[1. 0.] target=[0.] (реальный=-2)
A= -2 B= 5 -> вход=[0. 1.] target=[0.] (реальный=-2)
A= -2 B= -2 -> вход=[0. 0.] target=[1.] (реальный=5)

=====
Серия из 5 запусков – Конфигурация А (MSE)
=====

Результаты (MSE):
seed    epochs    converged    Es    accuracy    MAE
1        2764        True        0.009992    1.00    0.3482
2       10000        False       0.517764    0.75    1.8124
3        2761        True        0.009992    1.00    0.3471
4        2907        True        0.009992    1.00    0.3487
5        3447        True        0.009995    1.00    0.3464

--- Демонстрация работы сети «Конфигурация А (MSE)» ---
На 4 примерах обучающей выборки:
target=5      A= 5 B= 5 | вход=[1.0, 1.0] | y_hat=0.9555 | y_real= 4.688 | ближе к классу c0 (5)
target=-2     A= 5 B=-2 | вход=[1.0, 0.0] | y_hat=0.0586 | y_real=-1.590 | ближе к классу c1 (-2)
target=-2     A=-2 B= 5 | вход=[0.0, 1.0] | y_hat=0.0452 | y_real=-1.684 | ближе к классу c1 (-2)
target=5      A=-2 B=-2 | вход=[0.0, 0.0] | y_hat=0.9499 | y_real= 4.650 | ближе к классу c0 (5)
На дополнительных парах (вне обучающей выборки):
вне выборки   A= 3 B= 7 | вход=[1.0, 1.0] | y_hat=0.9555 | y_real= 4.688 | ближе к классу c0 (5)
вне выборки   A=-5 B=-8 | вход=[0.0, 0.0] | y_hat=0.9499 | y_real= 4.650 | ближе к классу c0 (5)
вне выборки   A= 0 B= 0 | вход=[0.0, 0.0] | y_hat=0.9499 | y_real= 4.650 | ближе к классу c0 (5)

--- Демонстрация работы сети «Конфигурация Б (BCE)» ---
На 4 примерах обучающей выборки:
target=5      A= 5 B= 5 | вход=[1.0, 1.0] | y_hat=0.9897 | y_real= 4.928 | ближе к классу c0 (5)
target=-2     A= 5 B=-2 | вход=[1.0, 0.0] | y_hat=0.0166 | y_real=-1.884 | ближе к классу c1 (-2)
target=-2     A=-2 B= 5 | вход=[0.0, 1.0] | y_hat=0.0105 | y_real=-1.926 | ближе к классу c1 (-2)
target=5      A=-2 B=-2 | вход=[0.0, 0.0] | y_hat=0.9881 | y_real= 4.917 | ближе к классу c0 (5)
На дополнительных парах (вне обучающей выборки):
вне выборки   A= 3 B= 7 | вход=[1.0, 1.0] | y_hat=0.9897 | y_real= 4.928 | ближе к классу c0 (5)
вне выборки   A=-5 B=-8 | вход=[0.0, 0.0] | y_hat=0.9881 | y_real= 4.917 | ближе к классу c0 (5)
вне выборки   A= 0 B= 0 | вход=[0.0, 0.0] | y_hat=0.9881 | y_real= 4.917 | ближе к классу c0 (5)

Выберите сеть для интерактивного режима:
1 - Конфигурация А (MSE)
2 - Конфигурация Б (BCE)
Ваш выбор (1/2, Enter для пропуска): 1

--- Режим работы обученной сети (Конфигурация А (MSE)) ---
Введите пару чисел А, В из диапазона [-10, 10] (или 'q' для выхода).
Введите А В через пробел: -4 7
      ввод пользователя      A= -4.0 B= 7.0 | вход=[0.0, 1.0] | y_hat=0.0452 | y_real=-1.684 | ближе к классу c1 (-2)
Введите А В через пробел:
Завершение режима ввода.
```

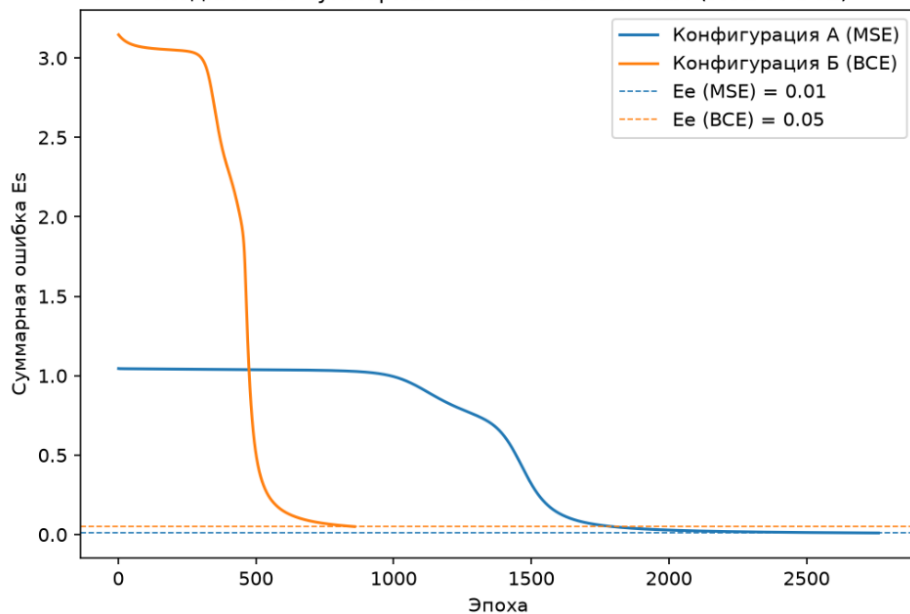
```
=====
Серия из 5 запусков – Конфигурация Б (BCE)
=====

Результаты (BCE):
seed    epochs    converged    Es    accuracy    MAE
1        10000        False       1.525147    0.75    1.7513
2        10000        False       1.525169    0.75    1.7514
3         858         True        0.049997    1.00    0.0862
4         896         True        0.049981    1.00    0.0862
5        1230         True        0.049911    1.00    0.0858

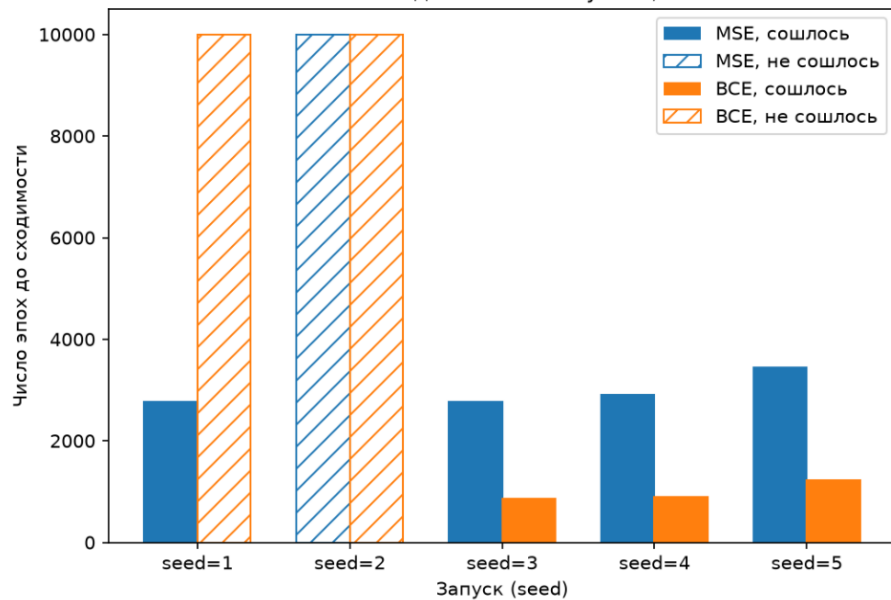
=====
Сводка по устойчивости сходимости
=====
MSE: сошло 4/5 запусков; эпох (сошедшиеся): [2764, 2761, 2907, 3447] (среднее=2969.8)
BCE: сошло 3/5 запусков; эпох (сошедшиеся): [858, 896, 1230] (среднее=994.7)

=====
Итоговая таблица (представительные запуски: MSE seed=3, BCE seed=3)
=====
Критерий                                     Конфиг. А (MSE)    Конфиг. Б (BCE)
Эпох до сходимости                           2761              858
Критерий Es<=Ee достигнут                     True              True
Итоговая Es                                   0.009992          0.049997
Accuracy                                       1.00              1.00
MAE (шкала c0/c1)                             0.3471            0.0862
```

Сходимость: суммарная ошибка E_s по эпохам (MSE vs BCE)



Устойчивость сходимости: 5 запусков, MSE vs BCE



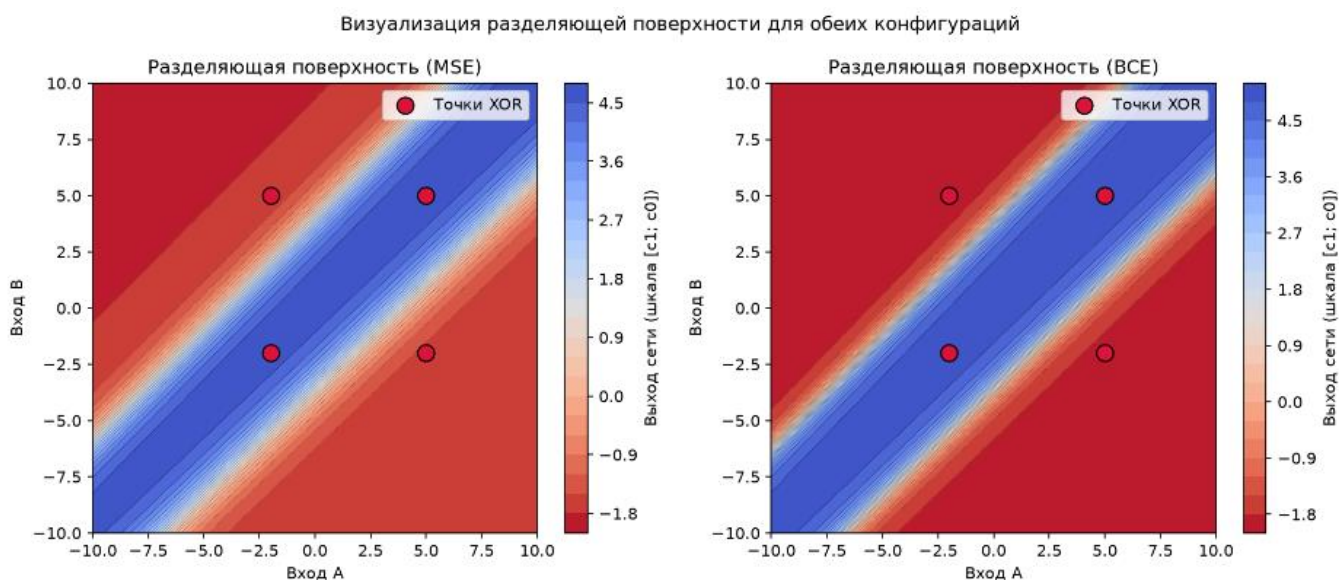
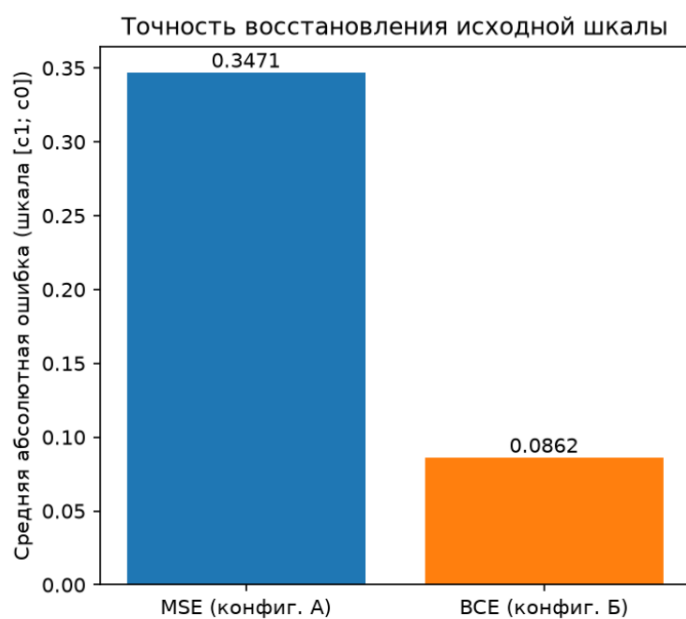


Таблица результатов обучения

Серия из 5 запусков — Конфигурация А (MSE), $E_e = 0.01$

seed	эпох	сошлось	итог. Es	accuracy	MAE
1	2764	Да	0.009992	1.00	0.3482
2	10000	Нет	0.517764	0.75	1.8124
3	2761	Да	0.009992	1.00	0.3471
4	2907	Да	0.009992	1.00	0.3487
5	3447	Да	0.009995	1.00	0.3464

Серия из 5 запусков — Конфигурация Б (BCE), $E_e = 0.05$

seed	эпох	сошлось	итог. Es	accuracy	MAE
1	10000	Нет	1.525147	0.75	1.7513
2	10000	Нет	1.525169	0.75	1.7514
3	858	Да	0.049997	1.00	0.0862
4	896	Да	0.049981	1.00	0.0862

5	1230	Да	0.049911	1.00	0.0858
---	------	----	----------	------	--------

Сводная таблица (представительные запуски: MSE seed=3, BCE seed=3)

Критерий	Конфиг. А (MSE)	Конфиг. Б (BCE)
Эпох до сходимости	2761	858
Критерий $E_s \leq E_e$ достигнут	Да	Да
Итоговая E_s	0.009992	0.049997
Accuracy	1.00	1.00
MAE (шкала c0/c1)	0.3471	0.0862
Сошлось запусков из 5	4 из 5	3 из 5
Среднее число эпох (сошедшиеся)	2969.8	994.7

Результаты работы обученной сети

Конфигурация А (MSE):

А	В	Тип примера	$\hat{y}$	y_real	Класс
5	5	обучающий (target=5)	0.9555	4.688	c0 (5)
5	-2	обучающий (target=-2)	0.0586	-1.590	c1 (-2)
-2	5	обучающий (target=-2)	0.0452	-1.684	c1 (-2)
-2	-2	обучающий (target=5)	0.9499	4.650	c0 (5)
3	7	вне выборки	0.9555	4.688	c0 (5)
-5	-8	вне выборки	0.9499	4.650	c0 (5)
0	0	вне выборки	0.9499	4.650	c0 (5)

Конфигурация Б (BCE):

А	В	Тип примера	$\hat{y}$	y_real	Класс
5	5	обучающий (target=5)	0.9897	4.928	c0 (5)
5	-2	обучающий (target=-2)	0.0166	-1.884	c1 (-2)
-2	5	обучающий (target=-2)	0.0105	-1.926	c1 (-2)
-2	-2	обучающий (target=5)	0.9881	4.917	c0 (5)
3	7	вне выборки	0.9897	4.928	c0 (5)
-5	-8	вне выборки	0.9881	4.917	c0 (5)
0	0	вне выборки	0.9881	4.917	c0 (5)

Все примеры (включая примеры вне обучающей выборки) классифицированы верно обеими конфигурациями. Заметно, что выходы BCE (0.9897/0.0105...) ближе к крайним точкам шкалы 0/1, чем выходы MSE (0.9555/0.0452...) — сеть, обученная с BCE, «увереннее» в своих ответах, что напрямую отражается в её более низкой MAE.

1. Какая функция потерь обеспечила более быструю и/или устойчивую сходимость?

По скорости — однозначно BCE: представительные запуски сходятся за ~860–1230 эпох против ~2760–3450 у MSE, то есть примерно в 2.5–3 раза быстрее. По устойчивости результат смешанный: из 5 запусков MSE сошёлся в 4 случаях, BCE — в 3; то есть BCE не показал большей устойчивости к неудачной инициализации, хотя теоретически ожидалось обратное. Скорость сходимости согласуется с известным теоретическим преимуществом BCE (устранение насыщения сигмоиды на выходном слое — см. раздел 2.3), а устойчивость к начальным весам определяется скорее структурой локальных минимумов конкретной задачи, чем типом функции потерь.

2. Как нормализация целевых значений повлияла на процесс обучения?

Нормализация $c_0/c_1 \rightarrow 0/1$ применена одинаково в обеих конфигурациях (и уже использовалась в ЛР №1), поэтому сама по себе не является переменной сравнения в этой работе — без неё сигмоидный выходной слой вообще не мог бы корректно обучаться. Нормализация не влияет на скорость/стабильность сама по себе — она лишь переводит задачу в область значений, где сигмоида и её производная работают корректно; на скорость и стабильность влияет именно выбор функции потерь, применённой уже к нормализованным меткам.

3. Точность восстановления исходной шкалы (MAE): BCE против MSE?

Точность оказалась заметно выше у BCE: $MAE \approx 0.086$ против $MAE \approx 0.347$ у MSE (шкала $[c_1; c_0]$) — примерно в 4 раза точнее. Явного компромисса «удобство классификации vs точность регрессии» не наблюдается — BCE выигрывает по обоим показателям сразу, поскольку отсутствие насыщения градиента позволяет сети дольше «дожимать» выход к крайним значениям 0/1, что напрямую повышает точность обратного пересчёта в реальную шкалу.

4. Насколько устойчивы обе конфигурации к случайной инициализации весов?

Обе конфигурации показали неполную устойчивость: MSE сошёлся в 4 из 5 запусков, BCE — в 3 из 5. В обоих случаях наблюдались запуски, «застрявшие» в локальном минимуме — суммарная ошибка переставала уменьшаться и не достигала порога E_{ϵ} за 10000 эпох, при этом ассигасу в таких случаях падала до 0.75 (сеть верно решала только 3 из 4 примеров). Это подтверждает, что качество обучения MLP для XOR существенно зависит от случайной инициализации весов, независимо от типа функции потерь.

5. Подтвердилось ли наблюдение ЛР №1 о линейной неразделимости задачи независимо от конкретных c_0, c_1 ?

Да, полностью подтвердилось. Структура задачи (совпадающие входы $\rightarrow$ один класс, несовпадающие $\rightarrow$ другой) не зависит от конкретных числовых значений $c_0=5, c_1=-2$ — после бинарного кодирования два класса расположены по диагоналям квадрата и неразделимы одной прямой. Все выводы ЛР №1 остаются в силе и в ЛР №2, независимо от смены функции потерь.

6. Как соотносится выбор функции потерь с необходимостью многослойной архитектуры для XOR?

Тип функции потерь никак не влияет на этот вывод — он определяется исключительно архитектурой сети. И MSE, и BCE применялись к одной и той же архитектуре MLP 2-2-1, и обе успешно решили задачу при удачной инициализации весов. Функция потерь влияет только на скорость и устойчивость обучения, но не на принципиальную возможность решения задачи — эта возможность определяется наличием скрытого слоя, формирующего нелинейное разбиение пространства признаков.

7. Что показывает визуализация разделяющей поверхности?

Визуализация показывает для обеих конфигураций одинаковую по форме диагональную структуру, соответствующую ожидаемой для XOR картине «пересечения двух полуплоскостей», формируемых скрытым слоем: высокие значения (класс c_0) сосредоточены вдоль диагонали, проходящей через точки $(5,5)$ и $(-2,-2)$, низкие значения (класс c_1) — вдоль противоположной области, где расположены точки $(5,-2)$ и $(-2,5)$. Визуально между конфигурацией А (MSE) и конфигурацией Б (BCE) форма границы практически не отличается — различается только резкость перехода между зонами, что согласуется с более высокой уверенностью (меньшим MAE) сети, обученной с BCE.

Вывод: На основе многослойного персептрона архитектуры 2-2-1, реализованного в Лабораторной работе №1 для задачи XOR с числовыми классами c_0 и c_1 из диапазона $[-10; 10]$, реализовала альтернативную функцию потерь – бинарную кросс-энтропию (Binary Cross-Entropy, BCE) – и провести сравнительный анализ обучения сети с MSE (Лабораторная работа №1) и с BCE по скорости сходимости, качеству обучения и устойчивости к выбору начальных весов.